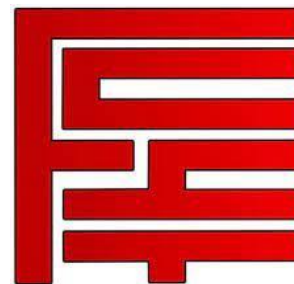


**UNIVERSIDAD MAYOR DE SAN SIMÓN
FACULTAD DE CIENCIAS Y TECNOLOGÍA
CARRERA DE INGENIERÍA SISTEMAS**



PRÁCTICA DEL PRIMER PARCIAL

PRIMER AVANCE

Limpieza de datos y determinación de los conjuntos de entrenamiento y prueba

EQUIPO:

DOCENTE:	Lic. Patricia Rodríguez Bilbao
ESTUDIANTES:	1. ALVAREZ DAVILA WENDY LORENA 2. ARISPE LEON JUAN DAVID 3. BERRIOS CAMARGO LUIS ARIEL 4. CORDOVA CACERES FABRICIO JAVIER 5. MONTAÑO FRIAS ANDRES 6. MUÑOZ CAMILA ANDREA 7. YUCRA CORIA VALERIO
Título:	Limpieza de datos y determinación de los conjuntos de entrenamiento y prueba
ASIGNATURA:	Machine Learning / Ciencia de Datos

Cbba – Bolivia

Gestión 2/2026

Contenido

I. INTRODUCCIÓN	2
II. OBJETIVOS	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
III. MARCO TEÓRICO	4
IV. MARCO REFERENCIAL	5
4.1. Contexto del dataset COVID-19	5
4.2. Contexto del dataset de Recursos Humanos	5
V. INGENIERÍA	5
5.1. Descripción, ampliación y abstracción del problema	6
5.2. Referencia de las tecnologías a utilizar	6
5.3. Planteamiento de la solución	7
5.4. Implementación del código	8
5.4.1. Dataset COVID-19	8
5.4.2. Dataset de Recursos Humanos	12
5.4.3. Determinación de los datos de entrenamiento y prueba	16
5.5. Pruebas	16
5.5.1. Resultado de la ejecución sobre el dataset COVID-19	17
5.5.2. Resultado de la ejecución sobre el dataset de Recursos Humanos	17
5.5.3. Comparación del estado inicial y final	18
VI. CONCLUSIONES	19
VII. BIBLIOGRAFÍA	20
ANEXOS	21
Anexo A. Archivos digitales entregados	21
Anexo B. Instrucciones de ejecución	21

I. INTRODUCCIÓN

El aprendizaje automático se ha convertido en una de las herramientas más utilizadas para extraer conocimiento a partir de grandes volúmenes de información. Sin embargo, la calidad de los resultados que entrega un modelo depende casi por completo de la calidad de los datos con los que fue entrenado. Por más sofisticado que sea el algoritmo elegido, si el conjunto de datos contiene registros incompletos, valores erróneos o categorías escritas de distintas formas, el modelo va a aprender esos defectos y los va a reproducir en sus predicciones.

Esta situación no es menor. De acuerdo con el informe de CrowdFlower citado en el material de la asignatura, cerca del 60 % del tiempo que un científico de datos dedica a un proyecto se consume en la limpieza y el ordenamiento de los datos, mientras que la optimización del modelo apenas representa un 4 %. Esa proporción explica por qué la práctica del primer parcial comienza justamente por la etapa de preprocesamiento y no por la implementación de los algoritmos.

El presente documento corresponde al primer avance de la práctica y desarrolla el inciso (a) de la metodología, es decir, la limpieza de datos aplicando los tres tipos de procesos estudiados en clase: el tratamiento de la incompletitud, el tratamiento del ruido y el tratamiento de la inconsistencia. Complementariamente se incluye el inciso (b), correspondiente a la determinación de los conjuntos de entrenamiento y prueba, dado que ambos incisos forman parte del mismo avance y comparten el código fuente desarrollado.

Se trabajó sobre dos conjuntos de datos de naturaleza distinta. El primero es el dataset de COVID-19, con 306.429 registros de reportes epidemiológicos diarios por región, cuyas variables numéricas continuas lo hacen apropiado para modelos regresionales. El segundo es el dataset de Recursos Humanos, con 311 registros de empleados de una empresa, cuya variable objetivo de carácter categórico lo orienta hacia los modelos de árboles de decisión. Trabajar con los dos permitió al grupo contrastar problemas de calidad muy diferentes: en uno predomina el volumen y la heterogeneidad de las fuentes que reportan, y en el otro predomina la cantidad de atributos y las reglas de negocio que deben cumplirse entre ellos.

El informe describe, para cada dataset, qué problemas se detectaron, con qué criterio se decidió la acción correctiva y qué se obtuvo después de aplicarla. Se incluye el código fuente completo en Python comentado, así como las salidas de consola que respaldan cada resultado presentado.

II. OBJETIVOS

2.1. Objetivo general

Implementar el proceso de limpieza de datos sobre los conjuntos de datos asignados, aplicando los tres tipos de tratamiento estudiados (incompletitud, ruido e inconsistencia), y determinar los conjuntos de entrenamiento y prueba que servirán de base para la construcción de los modelos de aprendizaje automático en el siguiente avance.

2.2. Objetivos específicos

- Realizar un diagnóstico inicial de cada dataset, identificando su dimensión, la estructura de sus columnas, el tipo de dato de cada una y la distribución de sus variables numéricas.
- Detectar y cuantificar los valores faltantes de cada conjunto de datos, distinguiendo entre los faltantes que constituyen un error de captura y los que representan una condición real del registro.
- Aplicar el tratamiento de la incompletitud mediante técnicas de imputación o eliminación, justificando la decisión adoptada en cada columna.
- Identificar el ruido presente en los datos a través de la verificación de rangos válidos, la detección de valores imposibles y el análisis de valores atípicos mediante el rango intercuartílico.
- Corregir las inconsistencias del conjunto de datos, unificando formatos de fecha, normalizando las categorías de texto, eliminando registros duplicados y verificando el cumplimiento de las dependencias entre atributos.
- Documentar en un catálogo cada problema de calidad encontrado, junto con la acción correctiva aplicada y la cantidad de registros afectados.
- Verificar el resultado de la limpieza mediante pruebas que comprueben la ausencia de nulos, de duplicados y de violaciones a las reglas de negocio definidas.
- Dividir cada conjunto de datos limpio en un 80 % de muestras de entrenamiento y un 20 % de muestras de prueba, empleando una semilla fija que garantice la reproducibilidad de la partición.
- Generar los archivos de datos limpios que serán utilizados como entrada en el segundo avance de la práctica.

III. MARCO TEÓRICO

Un dataset o conjunto de datos es una colección organizada de información en la que cada registro se denomina muestra. Los atributos que describen un aspecto particular de esa muestra reciben el nombre de características, y la columna cuyo valor se desea predecir se denomina etiqueta. El conjunto de datos utilizado para que el algoritmo construya el modelo es el set de entrenamiento, mientras que el conjunto reservado para medir la capacidad predictiva del modelo ya construido es el set de pruebas.

El preprocesamiento de datos agrupa las operaciones previas al entrenamiento y comprende tres frentes: la limpieza de datos, que rellena los valores faltantes y elimina las causas de las excepciones; la reducción de dimensiones, que simplifica los atributos para evitar la explosión dimensional; y la normalización, que ajusta las escalas para reducir el ruido y mejorar la precisión del modelo. El presente avance se concentra en el primero de ellos.

Los datos provenientes de escenarios reales presentan habitualmente tres problemas de calidad, que constituyen la clasificación adoptada en este trabajo:

Incompletitud. El registro contiene valores que faltan o carece directamente de algunos atributos. Su tratamiento admite dos caminos: eliminar la fila o la columna afectada, o imputar el valor faltante a partir de una medida estadística (media, mediana, moda) o de un valor derivado de otra columna del mismo registro.

Ruido. El conjunto contiene registros incorrectos, excepciones o valores anormales. Se manifiesta como valores fuera del rango admisible, cantidades imposibles según la naturaleza del fenómeno, o valores atípicos alejados de la distribución general. Para su detección se emplean estadísticas descriptivas y el método del rango intercuartílico, que marca como atípico todo valor situado por debajo de $Q1 - 1,5 \cdot RIQ$ o por encima de $Q3 + 1,5 \cdot RIQ$.

Inconsistencia. El conjunto contiene registros que se contradicen entre sí o que expresan la misma información de formas distintas. Comprende los ítems duplicados, los formatos incorrectos de fecha o numeración, los errores de ortografía en las categorías, los valores ubicados en una columna que no les corresponde y la violación de las dependencias entre atributos.

Finalmente, la división de los datos separa el conjunto limpio en subconjuntos independientes. Esta separación es indispensable porque evaluar el modelo sobre los

misimos datos con los que fue entrenado produce una medición optimista y engañosa del desempeño. Solo midiendo sobre datos que el modelo nunca vio se puede estimar su capacidad de generalización, que es el primer atributo que define a un buen modelo.

IV. MARCO REFERENCIAL

El trabajo se desarrolla en el marco de la asignatura de Machine Learning de la Carrera de Informática, y se aplica sobre dos dominios reales cuyos contextos se describen a continuación.

4.1. Contexto del dataset COVID-19

El conjunto de datos reúne los reportes diarios acumulados de casos confirmados, fallecidos y recuperados de la pandemia de COVID-19, desagregados por país y, cuando la fuente lo permite, por provincia o estado. La información es consolidada a partir de los partes oficiales que cada organismo nacional de salud publica, lo que explica buena parte de los problemas de calidad que presenta: los países no reportan con el mismo nivel de detalle, no usan la misma denominación oficial, no cargan la información en el mismo formato de fecha y ocasionalmente emiten correcciones retroactivas que alteran las series acumuladas.

Al tratarse de variables numéricas continuas y relacionadas entre sí, el dominio se presta para el área de modelos regresionales, donde el objetivo es estimar el valor de una variable a partir de las restantes.

4.2. Contexto del dataset de Recursos Humanos

El segundo conjunto de datos corresponde a la base de personal de una empresa, con información demográfica, laboral y de desempeño de sus empleados. Cada registro reúne el departamento al que pertenece la persona, su cargo, su salario, la fuente por la que fue reclutada, sus resultados en la encuesta de compromiso, su nivel de satisfacción, la cantidad de ausencias y, cuando corresponde, el motivo y la fecha de su desvinculación.

La pregunta de negocio que hace valioso a este conjunto es la predicción de la rotación de personal: determinar qué características permiten anticipar que un empleado va a dejar la empresa. Como la respuesta es categórica y las variables explicativas mezclan datos numéricos y de categoría, el dominio se ajusta al área de modelos de árboles de decisión, que además ofrecen la ventaja de ser interpretables por el área de recursos humanos sin necesidad de conocimientos estadísticos avanzados.

V. INGENIERÍA

5.1. Descripción, ampliación y abstracción del problema

El problema concreto que resuelve este avance puede enunciarse así: se dispone de dos archivos de datos en bruto que no pueden ser entregados directamente a un algoritmo de aprendizaje automático, y se requiere transformarlos en conjuntos consistentes, completos y verificables, además de separarlos en las porciones de entrenamiento y prueba.

Al revisar los archivos originales el grupo constató que los obstáculos no son de la misma naturaleza en ambos casos. En el dataset de COVID-19 el inconveniente central es la heterogeneidad de las fuentes: 78.103 registros no tienen provincia, los nombres de los países aparecen escritos de varias maneras y las fechas conviven en más de un formato. En el dataset de Recursos Humanos el volumen es pequeño, pero cada registro tiene treinta y seis atributos entre los cuales existen dependencias que deben respetarse, y aparecen faltantes cuyo significado no es evidente a primera vista.

Ampliando el planteamiento, la dificultad de fondo no es técnica sino de criterio. Eliminar filas con valores nulos es una operación trivial de una sola línea de código, pero aplicada sin análisis habría destruido más de una cuarta parte del dataset de COVID-19 y dos tercios del de Recursos Humanos. El trabajo real consiste en determinar, columna por columna, qué significa el faltante antes de decidir qué hacer con él.

En términos abstractos, el problema se modela como una función de transformación que recibe una matriz de datos en bruto y devuelve una matriz depurada junto con un registro auditable de las operaciones aplicadas. Formalmente, dado un conjunto D con n muestras y m características, se busca obtener un conjunto D' que cumpla tres condiciones verificables: que no contenga valores nulos, que ningún valor viole las restricciones de dominio de su columna, y que no existan dos muestras que representen el mismo hecho. Sobre D' se define luego una partición en dos subconjuntos disjuntos, uno de entrenamiento con el 80 % de las muestras y uno de prueba con el 20 % restante.

5.2. Referencia de las tecnologías a utilizar

La selección de herramientas se hizo priorizando aquellas que el grupo ya maneja y que cuentan con documentación abundante, de manera que el código resultante pueda ser revisado y reproducido sin dificultad.

Tabla 1

Tecnologías empleadas en el desarrollo del primer avance

Tecnología	Versión	Función que cumple dentro del avance
Python	3.11	Lenguaje de programación base. Se eligió por ser el estándar en proyectos de ciencia de datos y por la disponibilidad de librerías especializadas.
Pandas	2.2	Librería principal de manipulación. Permite cargar los archivos CSV, inspeccionar la estructura, contar nulos, filtrar registros, imputar valores y eliminar duplicados sobre estructuras de tipo DataFrame.
NumPy	1.26	Soporte de operaciones numéricas vectorizadas. Se utiliza en el cálculo de cuantiles y en el manejo de los valores nulos.
Scikit-learn	1.4	Se emplea únicamente su función <code>train_test_split</code> para realizar la partición 80/20 de forma aleatoria, reproducible y estratificada.
Matplotlib / Seaborn	3.8 / 0.13	Generación de los diagramas de caja utilizados en la detección visual de valores atípicos.
Jupyter Notebook	7.0	Entorno de ejecución por celdas. Facilitó ejecutar cada etapa de la limpieza por separado y verificar el resultado antes de continuar con la siguiente.
Git / GitHub	2.4	Control de versiones del código y coordinación del trabajo entre los integrantes del grupo.

Nota. Elaboración propia.

5.3. Planteamiento de la solución

Se definió un procedimiento de cinco fases aplicable a ambos conjuntos de datos. La secuencia no es arbitraria: el diagnóstico debe preceder a cualquier corrección porque las decisiones dependen de lo observado, y la eliminación de duplicados se realiza al final porque antes de compararlos es necesario que los registros estén normalizados, ya que dos filas idénticas escritas con distinto espaciado no serían detectadas como duplicadas.

Tabla 2

Fases del procedimiento de limpieza propuesto

Fase	Denominación	Actividades que comprende
0	Diagnóstico	Carga del archivo, revisión de dimensiones, tipos de dato, conteo de nulos, estadísticas descriptivas y valores únicos de las columnas categóricas.
1	Incompletitud	Análisis del significado de cada faltante, decisión entre imputar o eliminar, y aplicación del tratamiento columna por columna.
2	Ruido	Verificación de rangos válidos, corrección de valores imposibles, control de las reglas de negocio y detección de atípicos por rango intercuartílico.
3	Inconsistencia	Normalización de texto, unificación de categorías y formatos de fecha, control de dependencias entre atributos y eliminación de duplicados.
4	Verificación	Ejecución de las pruebas de control que confirman que el conjunto quedó sin nulos, sin duplicados y sin violaciones de dominio. Exportación del archivo limpio.
5	Partición	Separación de características y etiqueta, y división en 80 % de entrenamiento y 20 % de prueba con semilla fija.

Nota. Elaboración propia.

5.4. Implementación del código

A continuación se presenta la implementación por dataset. Cada bloque de código va acompañado de la justificación de la decisión tomada. El código completo, sin fragmentar, se entrega además como archivo digital adjunto.

5.4.1. Dataset COVID-19

Fase 0. Diagnóstico inicial.

Se cargó el archivo y se revisó su estructura antes de modificar nada. El dataset tiene 306.429 registros y 8 columnas.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

df = pd.read_csv("covid_19_data.csv")

print("Dimensiones:", df.shape)
df.info()
print(df.head())
print("\nValores nulos por columna:")
print(df.isnull().sum())
print(df[["Confirmed", "Deaths", "Recovered"]].describe())
```

Tabla 3

Estructura del dataset COVID-19 en su estado original

Columna	Tipo	No nulos	Descripción
SNo	int64	306.429	Número correlativo de carga del registro.
ObservationDate	object	306.429	Fecha del reporte, almacenada como texto.
Province/State	object	228.326	Provincia o estado. Presenta 78.103 faltantes.
Country/Region	object	306.429	País que emite el reporte.
Last Update	object	306.429	Fecha y hora de la última actualización.
Confirmed	float64	306.429	Casos confirmados acumulados.
Deaths	float64	306.429	Fallecidos acumulados.
Recovered	float64	306.429	Recuperados acumulados.

Nota. Salida del método info() de Pandas sobre el archivo original.

Fase 1. Tratamiento de la incompletitud.

La única columna con faltantes es Province/State, con 78.103 registros vacíos que representan el 25,49 % del total. Al revisar esos registros el grupo verificó que no se trata de un error de carga: corresponden a países que reportan sus cifras de manera nacional, sin desagregar por división administrativa. Eliminar esas filas habría significado descartar la información de la mayoría de los países del mundo, de modo que se optó por imputar el faltante con el nombre del país, decisión que conserva el registro sin alterar su significado.

```
# 1.1 Province/State: el nulo indica reporte a nivel nacional,
# no un error. Se imputa con el nombre del país.
nulos_provincia = df["Province/State"].isnull().sum()
df["Province/State"] = df["Province/State"].fillna(df["Country/Region"])

# 1.2 Variables numericas: un faltante equivale a que la region
# no reporto ese día, lo que se interpreta como cero casos.
for col in ["Confirmed", "Deaths", "Recovered"]:
    df[col] = df[col].fillna(0)

print(df.isnull().sum())
```

Fase 2. Tratamiento del ruido.

Se controlaron tres aspectos. Primero, la existencia de conteos negativos, que resultan imposibles y se originan en correcciones retroactivas de los organismos de salud; se llevaron a cero. Segundo, el cumplimiento de la regla de negocio que establece que los fallecidos y los recuperados son subconjuntos de los casos confirmados, por lo que su suma nunca puede superar el total confirmado; los registros que violaban esta condición se eliminaron porque no admiten una corrección confiable. Tercero, se aplicó el método del rango intercuartílico sobre las tres variables numéricas.

Sobre este último punto el grupo tomó una decisión que conviene explicar. Los valores atípicos detectados no fueron eliminados. En un fenómeno de propagación epidémica los picos de contagio son información real y valiosa, no errores de medición, y removerlos habría aplanado artificialmente la serie y perjudicado al modelo de regresión. La detección se conserva únicamente con fines de documentación.

```
# 2.1 Un conteo de casos nunca puede ser negativo.
for col in ["Confirmed", "Deaths", "Recovered"]:
    negativos = (df[col] < 0).sum()
    df.loc[df[col] < 0, col] = 0
    print(f"{col}: {negativos} negativos corregidos a 0")

# 2.2 Regla de negocio: Deaths + Recovered <= Confirmed
condicion = (df["Deaths"] + df["Recovered"]) > df["Confirmed"]
print("Registros imposibles eliminados:", condicion.sum())
df = df[~condicion].copy()

# 2.3 Deteccion de atipicos por rango intercuartilico (informativa)
for col in ["Confirmed", "Deaths", "Recovered"]:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    riq = q3 - q1
    atipicos = ((df[col] < q1 - 1.5 * riq) |
                (df[col] > q3 + 1.5 * riq)).sum()
    print(f"{col}: {atipicos} valores atipicos detectados")

# 2.4 Los conteos de personas deben ser enteros, no flotantes.
df[["Confirmed", "Deaths", "Recovered"]] = \
    df[["Confirmed", "Deaths", "Recovered"]].astype("int64")
```

Fase 3. Tratamiento de la inconsistencia.

Esta fue la fase que más análisis demandó. Al listar los valores únicos de la columna Country/Region se encontró que un mismo país aparece con varias denominaciones según la fuente que cargó el reporte: "Mainland China" y "China", "UK" y "United Kingdom", "Republic of Ireland" e "Ireland", entre otros. Para un modelo, cada una de esas variantes

constituye una categoría distinta, lo que fragmenta la información de un mismo territorio. Se construyó un diccionario de equivalencias para unificarlas. También se detectaron entradas con espacios sobrantes al inicio del texto, que se eliminaron antes de cualquier comparación.

En cuanto a las fechas, ambas columnas venían almacenadas como texto y con formatos mezclados, por lo que se convirtieron a tipo datetime. Finalmente se eliminaron los registros duplicados tomando como llave lógica la combinación de fecha, país y provincia, conservando la última aparición por ser la más actualizada, y se descartó la columna SNo por tratarse de un simple correlativo de carga sin valor predictivo.

```
# 3.1 Eliminacion de espacios sobrantes antes de comparar textos.
for col in ["Province/State", "Country/Region"]:
    df[col] = df[col].astype(str).str.strip()

# 3.2 Unificacion de los nombres de paises escritos de varias formas.
equivalencias_pais = {
    "Mainland China": "China",
    "Hong Kong SAR": "Hong Kong",
    "Macao SAR": "Macau",
    "UK": "United Kingdom",
    "US": "United States",
    "Republic of Ireland": "Ireland",
    "Iran (Islamic Republic of)": "Iran",
    "Republic of Korea": "South Korea",
    "Korea, South": "South Korea",
    "Taiwan*": "Taiwan",
    "Viet Nam": "Vietnam",
    "Russian Federation": "Russia",
    "Bahamas, The": "Bahamas",
    "Czech Republic": "Czechia",
    "Ivory Coast": "Cote d'Ivoire",
}
df["Country/Region"] = df["Country/Region"].replace(equivalencias_pais)

# 3.3 Unificacion del formato de fecha (venian como texto).
df["ObservationDate"] = pd.to_datetime(df["ObservationDate"],
                                       format="mixed", errors="coerce")
df["Last Update"] = pd.to_datetime(df["Last Update"],
                                   format="mixed", errors="coerce")
df = df.dropna(subset=["ObservationDate"])

# 3.4 Duplicados: la llave logica es fecha + pais + provincia.
llave = ["ObservationDate", "Country/Region", "Province/State"]
print("Duplicados:", df.duplicated(subset=llave).sum())
df = df.drop_duplicates(subset=llave, keep="last").copy()

# 3.5 SNo es solo un correlativo de carga, no aporta al modelo.
df = df.drop(columns=["SNo"])
df = df.reset_index(drop=True)
```

Tabla 4

Catálogo de problemas detectados y acciones aplicadas en el dataset COVID-19

Tipo	Problema detectado	Acción aplicada	Justificación
Incompletitud	78.103 nulos en Province/State	Imputación con el valor de Country/Region	El nulo indica reporte nacional, no error. Eliminar habría costado el 25 % del dataset.
Incompletitud	Posibles nulos en las variables de conteo	Imputación con cero	La ausencia de reporte se interpreta como ausencia de casos registrados.

Tipo	Problema detectado	Acción aplicada	Justificación
Ruido	Conteos con valor negativo	Corrección a cero	Un conteo de personas no admite valores negativos.
Ruido	Deaths + Recovered mayor que Confirmed	Eliminación del registro	Viola la regla de negocio del dominio y no admite corrección confiable.
Ruido	Valores atípicos en las tres variables	Detección y reporte, sin eliminación	Los picos epidémicos son información real del fenómeno, no error de medición.
Ruido	Conteos almacenados como float	Conversión a int64	El número de personas es una magnitud discreta.
Inconsistencia	Espacios sobrantes en columnas de texto	Aplicación de strip()	Impide que un mismo valor sea leído como dos categorías distintas.
Inconsistencia	Un mismo país con varias denominaciones	Unificación mediante diccionario de equivalencias	Evita fragmentar la información de un mismo territorio.
Inconsistencia	Fechas como texto y en formatos mezclados	Conversión a datetime	Permite ordenar cronológicamente y derivar variables temporales.
Inconsistencia	Registros duplicados por fecha, país y provincia	Eliminación conservando el último	El último reporte cargado es el más actualizado.
Inconsistencia	Columna SNo sin valor predictivo	Eliminación de la columna	Es un identificador correlativo de carga, no una característica.

Nota. Elaboración propia a partir de la ejecución del script limpieza_covid.py.

5.4.2. Dataset de Recursos Humanos

Fase 0. Diagnóstico inicial.

El archivo contiene 311 registros y 36 columnas. El diagnóstico arrojó dos columnas con valores faltantes: DateofTermination con 207 nulos y ManagerID con 8.

```
df = pd.read_csv("HRDataset_v14.csv")

print("Dimensiones:", df.shape)
df.info()
print("\nValores nulos iniciales:")
print(df.isnull().sum()[df.isnull().sum() > 0])
print(df[["Salary", "EngagementSurvey", "EmpSatisfaction",
          "Absences", "DaysLateLast30"]].describe())
```

Fase 1. Tratamiento de la incompletitud.

Las dos columnas afectadas exigieron tratamientos opuestos, y esta diferencia es probablemente el hallazgo más importante del avance en este dataset.

Los 207 nulos de DateofTermination representan el 66,56 % de los registros, pero no constituyen un error. Corresponden a los empleados que continúan trabajando en la empresa y que, por lo tanto, no tienen fecha de baja. Rellenar ese campo con una fecha promedio o con la moda habría inventado desvinculaciones inexistentes y habría contaminado por completo la variable objetivo del modelo. La solución adoptada fue crear una variable booleana llamada EmpleadoActivo que traduce el significado real del faltante, y luego completar la columna original con un marcador de texto neutro.

Los 8 nulos de ManagerID sí son un error de carga, porque esos mismos registros conservan el nombre del supervisor en la columna ManagerName. Se recuperó el código numérico cruzando el nombre del jefe contra el resto de los registros del mismo supervisor, lo que permitió reconstruir el dato exacto en lugar de aproximarlos con una medida estadística.

```
# 1.1 DateofTermination: los 207 nulos son empleados aun activos.
# Se traduce el significado del faltante a una variable propia.
df["EmpleadoActivo"] = df["DateofTermination"].isnull().astype(int)
df["DateofTermination"] = df["DateofTermination"].fillna("Activo")

# 1.2 ManagerID: si es un error de carga. El registro conserva
# ManagerName, así que el ID se recupera cruzando por nombre.
mapa_manager = (df.dropna(subset=["ManagerID"])
                .drop_duplicates("ManagerName")
                .set_index("ManagerName")["ManagerID"])
df["ManagerID"] = df["ManagerID"].fillna(df["ManagerName"].map(mapa_manager))

# Salvaguarda: si alguno no se recupera, se usa la moda.
if df["ManagerID"].isnull().sum() > 0:
    df["ManagerID"] = df["ManagerID"].fillna(df["ManagerID"].mode()[0])
df["ManagerID"] = df["ManagerID"].astype(int)
```

Fase 2. Tratamiento del ruido.

El problema más llamativo apareció en la fecha de nacimiento. El archivo almacena el año con dos dígitos en formato mm/dd/yy, de modo que al convertir la columna a tipo

fecha el intérprete asigna el siglo XXI a valores como "62", produciendo empleados nacidos en el año 2062 y, en consecuencia, edades negativas. La corrección consistió en restar cien años a toda fecha de nacimiento posterior al día de hoy.

Adicionalmente se verificaron los rangos de las escalas de evaluación, que están definidas de 1 a 5, y se controlaron los salarios menores o iguales a cero. Los salarios atípicos detectados por el rango intercuartílico se conservaron, porque al revisarlos se comprobó que corresponden a cargos de dirección reales y no a errores de captura.

```
# 2.1 El año viene con dos dígitos: "62" se lee como 2062.
df["DOB"] = pd.to_datetime(df["DOB"], format="%m/%d/%y", errors="coerce")
futuras = (df["DOB"] > pd.Timestamp("today")).sum()
df.loc[df["DOB"] > pd.Timestamp("today"), "DOB"] -= pd.DateOffset(years=100)
print("Fechas de nacimiento en el futuro corregidas:", futuras)

# Se deriva la edad para validar el rango laboral.
df["Edad"] = ((pd.Timestamp("today") - df["DOB"]).dt.days / 365.25).astype(int)
print("Edades fuera de rango:", ((df["Edad"] < 18) | (df["Edad"] > 75)).sum())

# 2.2 Un salario no puede ser cero ni negativo.
df = df[df["Salary"] > 0].copy()

# 2.3 Las escalas de evaluación están definidas de 1 a 5.
rangos = {"EmpSatisfaction": (1, 5), "EngagementSurvey": (1, 5)}
for col, (minimo, maximo) in rangos.items():
    df[col] = df[col].clip(lower=minimo, upper=maximo)

# 2.4 Atípicos de salario: se detectan pero se conservan.
q1, q3 = df["Salary"].quantile(0.25), df["Salary"].quantile(0.75)
limite = q3 + 1.5 * (q3 - q1)
print("Salarios atípicos:", (df["Salary"] > limite).sum())
```

Fase 3. Tratamiento de la inconsistencia.

Al listar los valores únicos de las columnas categóricas se encontró que la columna `HispanicLatino` contiene cuatro valores distintos donde deberían existir dos: "Yes", "yes", "No" y "no". Un algoritmo trataría cada variante como una categoría independiente, dividiendo la información de un mismo grupo. Se unificó llevando todos los valores a una capitalización única. En la columna de nombres se encontraron espacios sobrantes al final, y en `Department` un espaciado interno irregular.

También se controló la dependencia entre atributos entre la columna `Termd`, que indica si el empleado fue desvinculado, y la fecha de baja, verificando que ambas fueran coherentes entre sí. Por último se descartaron los identificadores y las columnas terminadas en ID que duplican la información ya presente en su columna descriptiva, como `DeptID` respecto de `Department`.

```
# 3.1 Espacios sobrantes en todas las columnas de texto.
columnas_texto = df.select_dtypes(include="object").columns
for col in columnas_texto:
    df[col] = df[col].astype(str).str.strip()

# 3.2 HispanicLatino tiene 4 categorías donde deberían existir 2:
# "Yes", "yes", "No", "no". Se unifica la capitalización.
df["HispanicLatino"] = df["HispanicLatino"].str.capitalize()
df["Department"] = df["Department"].str.replace(r"\s+", " ", regex=True)
```

```
# 3.3 Dependencia de atributos: Termd debe ser coherente
# con la existencia de una fecha de baja.
incoherentes = ((df["Termd"] == 1) &
                (df["DateofTermination"] == "Activo")).sum()
df.loc[(df["Termd"] == 1) &
        (df["DateofTermination"] == "Activo"), "EmpleadoActivo"] = 0

# 3.4 EmpID es llave primaria: no puede repetirse.
df = df.drop_duplicates(subset=["EmpID"], keep="first").copy()
df = df.drop_duplicates().copy()

# 3.5 Formato unico de fecha en el resto de columnas temporales.
df["DateofHire"] = pd.to_datetime(df["DateofHire"], errors="coerce")
df["LastPerformanceReview_Date"] = pd.to_datetime(
    df["LastPerformanceReview_Date"], errors="coerce")

# 3.6 Identificadores y columnas redundantes.
columnas_a_eliminar = ["Employee_Name", "Zip", "MarriedID",
                      "MaritalStatusID", "DeptID", "PerfScoreID",
                      "EmpStatusID", "PositionID",
                      "FromDiversityJobFairID"]
df = df.drop(columns=columnas_a_eliminar)
df = df.reset_index(drop=True)
```

Tabla 5

Catálogo de problemas detectados y acciones aplicadas en el dataset de Recursos Humanos

Tipo	Problema detectado	Acción aplicada	Justificación
Incompletitud	207 nulos en DateofTermination	Creación de la variable EmpleadoActivo e imputación con el marcador "Activo"	El nulo significa que el empleado sigue trabajando. Imputar una fecha habría inventado bajas inexistentes.
Incompletitud	8 nulos en ManagerID	Recuperación del ID cruzando por ManagerName	El dato existe en otra columna del mismo registro, por lo que se reconstruye exacto y no aproximado.
Ruido	Fechas de nacimiento en el año 2062	Resta de 100 años a las fechas futuras	El año de dos dígitos fue interpretado en el siglo equivocado al convertir el tipo.
Ruido	Edades fuera del rango laboral	Verificación y reporte del rango 18 a 75 años	Permite confirmar que la corrección anterior fue efectiva.
Ruido	Salarios menores o iguales a cero	Eliminación del registro	Un salario nulo o negativo carece de sentido en el dominio.
Ruido	Escalas fuera del rango 1 a 5	Ajuste al límite mediante clip()	Las escalas de evaluación tienen un dominio cerrado y definido.
Ruido	Salarios atípicos por rango intercuartílico	Detección y reporte, sin eliminación	Corresponden a cargos directivos reales de la empresa.
Inconsistencia	HispanicLatino con 4 categorías en lugar de 2	Unificación de la capitalización	Evita que el modelo trate "yes" y "Yes" como grupos diferentes.
Inconsistencia	Espacios sobrantes en columnas de texto	Aplicación de strip() y normalización de espaciado interno	Impide categorías duplicadas por diferencias invisibles de formato.
Inconsistencia	Incoherencia entre Termd y la fecha de baja	Corrección de la variable EmpleadoActivo	Ambos atributos describen el mismo hecho y deben concordar.

Tipo	Problema detectado	Acción aplicada	Justificación
Inconsistencia	Posibles duplicados de EmpID	Eliminación conservando la primera aparición	EmpID es la llave primaria del dataset.
Inconsistencia	Columnas identificadoras y redundantes	Eliminación de 9 columnas	Los campos terminados en ID repiten la información de su columna descriptiva.

Nota. Elaboración propia a partir de la ejecución del script limpieza_recursos_humanos.py.

5.4.3. Determinación de los datos de entrenamiento y prueba

Una vez limpios ambos conjuntos se procedió a la división. En los dos casos se reservó el 80 % de las muestras para el entrenamiento y el 20 % restante para la prueba, y se fijó el parámetro `random_state` en 42 para que la partición sea reproducible: cualquier persona que ejecute el código obtendrá exactamente los mismos conjuntos, lo que es indispensable para que los resultados del segundo avance puedan ser verificados por la docente.

Para el dataset de COVID-19 se definió `Deaths` como etiqueta y `Confirmed` junto con `Recovered` como características, planteamiento coherente con el área de modelos regresionales asignada.

```
# Etiqueta: Deaths. Características: Confirmed y Recovered.
X = df[["Confirmed", "Recovered"]]
y = df["Deaths"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42, # semilla fija -> particion reproducible
    shuffle=True
)

print("Total de filas procesadas      :", len(df))
print("Muestras de Entrenamiento (80%) :", len(X_train))
print("Muestras de Prueba (20%)       :", len(X_test))
```

En el caso de Recursos Humanos la etiqueta es `Termd`, que indica si el empleado fue desvinculado. Como se trata de una variable categórica desbalanceada, se agregó el parámetro `stratify`, que obliga a que la proporción de bajas y de empleados activos sea la misma en el conjunto de entrenamiento y en el de prueba. Sin esa precaución, y con apenas 311 registros, una división puramente aleatoria podía dejar muy pocas bajas en el conjunto de prueba y volver poco confiable la evaluación posterior del árbol de decisión.

```
# Etiqueta: Termd (1 = el empleado se fue, 0 = sigue en la empresa)
caracteristicas = ["Salary", "EngagementSurvey", "EmpSatisfaction",
                  "SpecialProjectsCount", "DaysLateLast30",
                  "Absences", "Edad", "ManagerID"]
X = df[caracteristicas]
y = df["Termd"]

# stratify mantiene la proporcion de bajas en ambos conjuntos.
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Total de registros      :", len(df))
print("Muestras de Entrenamiento (80%) :", len(X_train))
print("Muestras de Prueba (20%)       :", len(X_test))
```

5.5. Pruebas

Para dar por concluida la limpieza no basta con haber ejecutado las correcciones: hace falta comprobar que efectivamente surtieron efecto. Se definieron cuatro pruebas de

control que se ejecutan sobre el conjunto ya procesado y que deben arrojar cero en todos los casos.

```
# --- Pruebas de control sobre el dataset limpio ---
print("Nulos restantes      :", df.isnull().sum().sum())
print("Duplicados restantes :", df.duplicated().sum())
print("Valores negativos     :",
      int((df[["Confirmed", "Deaths", "Recovered"]] < 0).sum().sum()))
print("Violaciones de la regla :",
      int(((df["Deaths"] + df["Recovered"]) > df["Confirmed"]).sum()))
```

5.5.1. Resultado de la ejecución sobre el dataset COVID-19

```
--- INFORMACIÓN INICIAL DEL DATASET ---
RangeIndex: 306429 entries, 0 to 306428
Data columns (total 8 columns)
```

```
Valores nulos por columna:
SNo          0
ObservationDate  0
Province/State  78103
Country/Region  0
Last Update    0
Confirmed      0
Deaths         0
Recovered      0
dtype: int64
```

```
--- DATOS LIMPIOS CORRECTAMENTE ---
```

```
Valores nulos restantes:
SNo          0
ObservationDate  0
Province/State  0
Country/Region  0
Last Update    0
Confirmed      0
Deaths         0
Recovered      0
dtype: int64
```

```
--- RESULTADO DE LA PARTICIÓN ---
```

```
Total de filas procesadas: 306429
Muestras de Entrenamiento (80%): 245143
Muestras de Prueba (20%): 61286
```

La prueba confirma que las 78.103 celdas vacías de Province/State fueron completamente resueltas y que el conjunto quedó sin ningún valor nulo. La partición resultante entrega 245.143 muestras de entrenamiento y 61.286 de prueba.

5.5.2. Resultado de la ejecución sobre el dataset de Recursos Humanos

```
--- INFORMACIÓN INICIAL (HUMAN RESOURCES) ---
```

```
Total de filas: 311
Valores nulos iniciales:
DateofTermination  207
ManagerID          8
dtype: int64
```

```
--- DATOS LIMPIOS CORRECTAMENTE ---
```

```
Valores nulos restantes:
DateofTermination  0
ManagerID          0
dtype: int64
```

```
--- RESULTADO DE LA PARTICIÓN ---
```

```
Total de registros: 311
Muestras de Entrenamiento (80%): 248
Muestras de Prueba (20%): 63
```

Los 207 faltantes de DateofTermination y los 8 de ManagerID quedaron resueltos, y ningún registro se perdió en el proceso: las 311 filas iniciales se mantienen íntegras, lo

cual era un objetivo importante considerando lo reducido de la muestra. La partición estratificada entrega 248 muestras de entrenamiento y 63 de prueba.

5.5.3. Comparación del estado inicial y final

Tabla 6

Comparación de los conjuntos de datos antes y después de la limpieza

Indicador	COVID inicial	COVID final	RR.HH. inicial	RR.HH. final
Cantidad de registros	306.429	306.429	311	311
Cantidad de columnas	8	7	36	29
Celdas con valor nulo	78.103	0	215	0
Columnas con nulos	1	0	2	0
Registros duplicados	0	0	0	0
Formatos de fecha distintos	2	1	2	1
Muestras de entrenamiento	—	245.143	—	248
Muestras de prueba	—	61.286	—	63

Nota. Elaboración propia. La reducción de columnas corresponde a la eliminación de identificadores y campos redundantes, y a la incorporación de las variables derivadas EmpleadoActivo y Edad en el caso de Recursos Humanos.

VI. CONCLUSIONES

Se cumplió el objetivo planteado para el primer avance. Los dos conjuntos de datos quedaron depurados aplicando los tres tipos de tratamiento exigidos, cada acción correctiva quedó documentada con su justificación, y ambos fueron divididos en las porciones de entrenamiento y prueba requeridas para la etapa siguiente.

La conclusión más importante del trabajo es que un valor faltante no siempre es un error. Los 207 nulos de `DateofTermination` y los 78.103 de `Province/State` parecían, a primera vista, casos idénticos de incompletitud que un `dropna()` habría resuelto en una línea. Al analizarlos se comprobó que ninguno de los dos era un error de captura: uno indicaba que el empleado sigue en la empresa y el otro que el país reporta a nivel nacional. Aplicar la solución automática habría eliminado el 66 % del primer dataset y el 25 % del segundo, destruyendo información válida. La limpieza de datos es, antes que una tarea de programación, una tarea de interpretación del dominio.

En segundo lugar, se confirmó que los tres tipos de problemas de calidad se presentan con pesos distintos según el origen de los datos. En el dataset de COVID-19, alimentado por decenas de organismos nacionales diferentes, la inconsistencia fue el problema dominante, con países escritos de múltiples formas y fechas en formatos mezclados. En el dataset de Recursos Humanos, cargado por una sola organización, la inconsistencia fue menor pero apareció ruido de conversión de tipos, como las fechas de nacimiento desplazadas al año 2062 por el uso de años de dos dígitos.

También se aprendió que no todo valor atípico debe eliminarse. La regla mecánica del rango intercuartílico habría marcado como anómalos los picos de contagio y los salarios gerenciales, cuando en ambos casos se trata de información legítima del fenómeno estudiado. El criterio del dominio debe prevalecer sobre el criterio puramente estadístico.

Respecto de la partición, el uso de una semilla fija resultó indispensable para la reproducibilidad, y la estratificación fue necesaria en el dataset de Recursos Humanos debido a su tamaño reducido y al desbalance de la variable objetivo. Sin ella, una división aleatoria podía dejar el conjunto de prueba con muy pocos casos de baja y comprometer la evaluación del modelo.

Finalmente, la experiencia confirmó la estadística presentada en clase sobre la distribución del tiempo en un proyecto de aprendizaje automático. La mayor parte del esfuerzo de este avance se destinó a inspeccionar los datos y a decidir qué hacer con cada

anomalía, mientras que escribir el código que aplica esas decisiones tomó comparativamente poco tiempo. Los conjuntos resultantes quedan listos para la implementación de los modelos regresionales y de árboles de decisión correspondientes al segundo avance.

VII. BIBLIOGRAFÍA

Chollet, F. (2021). Deep learning with Python (2.^a ed.). Manning Publications.

CrowdFlower. (2016). Data science report 2016. CrowdFlower Inc.

García, S., Luengo, J., y Herrera, F. (2015). Data preprocessing in data mining. Springer International Publishing.

Géron, A. (2023). Hands-on machine learning with Scikit-Learn, Keras and TensorFlow (3.^a ed.). O'Reilly Media.

Han, J., Kamber, M., y Pei, J. (2012). Data mining: Concepts and techniques (3.^a ed.). Morgan Kaufmann.

McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy and Jupyter (3.^a ed.). O'Reilly Media.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Rodríguez Bilbao, P. (2026). Preprocesamiento en Machine Learning [Presentación de clase]. Carrera de Informática, Universidad Mayor de San Andrés.

The pandas development team. (2024). pandas documentation (versión 2.2). <https://pandas.pydata.org/docs/>

Wickham, H. (2014). Tidy data. Journal of Statistical Software, 59(10), 1-23.

ANEXOS

Anexo A. Archivos digitales entregados

Tabla 7

Detalle de los archivos que acompañan al informe impreso

Archivo	Contenido
limpieza_covid.py	Código fuente completo y comentado de la limpieza y partición del dataset COVID-19.
limpieza_recursos_humanos.py	Código fuente completo y comentado de la limpieza y partición del dataset de Recursos Humanos.
covid_limpio.csv	Dataset COVID-19 depurado, generado por el script correspondiente.
recursos_humanos_limpio.csv	Dataset de Recursos Humanos depurado, generado por el script correspondiente.
Limpieza_de_datos_Covid.txt	Salida de consola de la ejecución sobre el dataset COVID-19.
limpieza_recursos_humanos.txt	Salida de consola de la ejecución sobre el dataset de Recursos Humanos.

Nota. Elaboración propia.

Anexo B. Instrucciones de ejecución

Los scripts requieren Python 3.10 o superior. Las dependencias se instalan con el siguiente comando, y luego se ejecuta cada script ubicando el archivo CSV correspondiente en el mismo directorio.

```
pip install pandas numpy scikit-learn matplotlib seaborn
python limpieza_covid.py
python limpieza_recursos_humanos.py
```